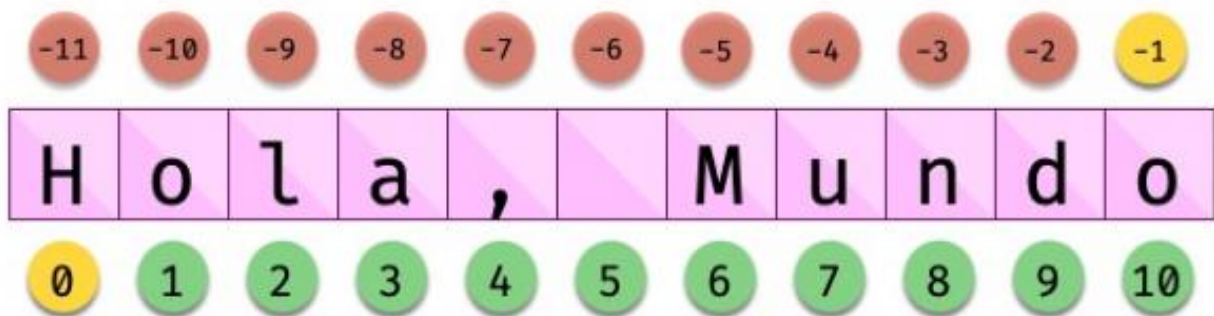


Cadenas de caracteres

Los «strings» están **indexados** y cada carácter tiene su propia posición. Para obtener un único carácter dentro de una cadena de texto es necesario especificar su **índice** dentro de corchetes [...].



Trocear una cadena

Es posible extraer «trozos» («rebanadas») de una cadena de texto. Tenemos varias aproximaciones para ello:

[:] Extrae la secuencia entera desde el comienzo hasta el final. Es una especie de copia de toda la cadena de texto.

[start:] Extrae desde start hasta el final de la cadena.

[start:end] Extrae desde el comienzo de la cadena hasta end menos 1.

[start:end] Extrae desde start hasta end menos 1.

[start:end:step] Extrae desde start hasta end menos 1 haciendo saltos de tamaño step.

Longitud de una cadena

Para obtener la longitud de una cadena podemos hacer uso de len(), una función común a prácticamente todos los tipos y estructuras de datos en Python:

```
>>> proverb = 'Lo cortés no quita lo valiente'
>>> len(proverb)
30

>>> empty = ''
>>> len(empty)
0
```

Si queremos comprobar que una determinada subcadena se encuentra en una cadena de texto

utilizamos el operador `in` para ello. Se trata de una expresión que tiene como resultado un valor «booleano» verdadero o falso:

```
proverb = Más vale malo conocido que bueno por conocer
```

```
"malo" in proverb
```

```
True
```

```
"bueno" in proverb
```

```
True
```

```
"regular" in proverb
```

```
False
```

Aunque hemos visto que la forma pitónica de saber si una subcadena se encuentra dentro

de otra es a través del operador `in`, Python nos ofrece distintas alternativas para realizar búsquedas en cadenas de texto.

Vamos a partir de una variable que contiene un trozo de la canción Mediterráneo de Joan

Manuel Serrat para ejemplificar las distintas opciones que tenemos:

```
lyrics = """Quizás porque mi niñez  
... Sigue jugando en tu playa  
... Y escondido tras las cañas  
... Duerme mi primer amor  
... Llevo tu luz y tu olor  
... Por dondequiera que vaya"""
```

Comprobar si una cadena de texto empieza o termina por alguna subcadena:

```
>>> lyrics.startswith('Quizás')
True

>>> lyrics.endswith('Final')
False
```

Encontrar la **primera ocurrencia** de alguna subcadena:

```
>>> lyrics.find('amor')
93

>>> lyrics.index('amor') # Same behaviour?
93
```

Tanto `find()` como `index()` devuelven el **índice** de la primera ocurrencia de la subcadena que estemos buscando, pero se diferencian en su comportamiento cuando la subcadena buscada no existe:

```
>>> lyrics.find('universo')
-1

>>> lyrics.index('universo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```

Contabilizar el **número de veces** que aparece una subcadena:

```
>>> lyrics.count('mi')
2

>>> lyrics.count('tu')
3

>>> lyrics.count('él')
0
```

Reemplazar elementos

Podemos usar la función `replace()` indicando la subcadena a reemplazar, la subcadena de reemplazo y cuántas instancias se deben reemplazar. Si no se especifica este último argumento, la sustitución se hará en todas las instancias encontradas:

```
>>> proverb = 'Quien mal anda mal acaba'

>>> proverb.replace('mal', 'bien')
'Quien bien anda bien acaba'

>>> proverb.replace('mal', 'bien', 1) # sólo 1 reemplazo
'Quien bien anda mal acaba'
```

Mayúsculas y minúsculas

Python nos permite realizar variaciones en los caracteres de una cadena de texto para pasarlos a mayúsculas y/o minúsculas. Veamos las distintas opciones disponibles:

```
>>> proverb = 'quien a buen árbol se arrima Buena Sombra le cobija'

>>> proverb
'quien a buen árbol se arrima Buena Sombra le cobija'

>>> proverb.capitalize()
'Quien a buen árbol se arrima buena sombra le cobija'

>>> proverb.title()
'Quien A Buen Árbol Se Arrima Buena Sombra Le Cobija'

>>> proverb.upper()
'QUIEN A BUEN ÁRBOL SE ARRIMA BUENA SOMBRA LE COBIJA'

>>> proverb.lower()
'quien a buen árbol se arrima buena sombra le cobija'

>>> proverb.swapcase()
'QUIEN A BUEN ÁRBOL SE ARRIMA buena sombra le COBIJA'
```